Program – M.Sc.IT (Faculty of Information, Communication and Technology)

Semester- 1

Course Code 266010344003	Name of Course Python Programming and Application Development	Compulsory
Credit: 03	Teaching Scheme: Theory (45) - Practical (0)	Teaching Hours: 45

Course Outcomes (COs)

After completing this course, students will be able to

CO-1: Apply core Python programming constructs and object-oriented principles - data types, control flow, functions, file handling and exception handling - to write modular, well-structured programs, and implement encapsulation, inheritance and polymorphism using classes and objects.

CO-2: Develop dynamic, database-driven web applications using the Flask framework by implementing routing, Jinja2 templates, forms, session management, CRUD operations with SQLite/MySQL, and basic REST APIs.

CO-3: Design, validate, document and test modern asynchronous REST APIs using the FastAPI framework, applying Pydantic models, dependency injection, authentication and automatic API documentation.

CO-4: Deploy and maintain Python web applications using professional practices including version control with Git, containerization with Docker, deployment with Gunicorn/Nginx, CI/CD pipelines, and responsible AI-assisted development.

Detailed Syllabus

Unit-1.

1.1 Introduction to Python: History, Features, Applications in Industry, Installing Python and VS Code, Interactive Mode vs Script Mode, Comments and Documentation,

1.2 Programming Constructs: Variables and Constants, Data Types, Type Casting, Input and Output, String Formatting (f-strings), Operators, Expressions.

1.3 Flow of Control: Decision Making statements: if, if-else, nested if, elif; Loop statements: for, while; break, continue, pass; enumerate, zip, range.

1.4 Built-in Data Structures: Strings, Lists, Tuples, Sets, Dictionaries, List / Dictionary / Set Comprehensions.

1.5 Functions: User-Defined Functions, Parameters and Return Values, *args and **kwargs, Variable Scope, Recursion, Lambda Functions, map and filter, Introduction to Decorators, Generators and Iterators.

1.6 Modules and Exception Handling: Creating and Importing Modules, Standard Library Overview; Exception Handling: try, except, finally, raise, User-Defined Exceptions; Type Hints and Annotations.

1.7 File Handling: Text Files, Context Managers (with statement), CSV Files, JSON Files.

1.8 Object-Oriented Programming: Classes and Objects, Constructors (__init__), Instance and Class Attributes, Encapsulation, Inheritance, Polymorphism, Dunder Methods (__str__, __repr__).

1.9 Environment and Tooling: pip, Virtual Environments (venv), requirements.txt

1.10 Problem Solving: Pattern Programs, Searching Techniques, Sorting Techniques,

Unit-2.

- **2.1 Database Fundamentals:** DBMS Concepts, SQLite, MySQL, CRUD Operations, Basic SQL.
- **2.2 Flask Framework:** Introduction to Python Web Frameworks and Their Applications, Installation and Project Structure, Routing, Dynamic Routes, Templates using Jinja2, Forms and Request Handling, Blueprints.
- **2.3 Database Integration:** SQLite with Flask, MySQL with Flask, Introduction to ORM (SQLAlchemy Basics).
- **2.4 Session and File Management:** Cookies, Sessions, File Uploading, Error Handling, Custom Error Pages.
- **2.5 REST API Fundamentals:** JSON, HTTP Methods: GET, POST, PUT, DELETE; Status Codes, Building a REST API in Flask, Testing with Postman Client.

Unit-3.

- **3.1 Introduction to FastAPI:** Comparison with Flask, Synchronous vs Asynchronous, Performance, Installation, Uvicorn, Project Structure.
- **3.2 Building APIs:** Path Parameters, Query Parameters, Request Bodies, Pydantic Models, Type Hints for Validation, Response Models, Status Codes.
- **3.3 Automatic Documentation:** Interactive Swagger UI, ReDoc, OpenAPI Specification.
- **3.4 Asynchronous Programming:** async and await, Async Endpoints, When to Use Async.
- **3.5 Data Layer:** Database Integration with SQL Alchemy, CRUD Service Layer, Dependency Injection (Depends).
- **3.6 Security and Middleware:** Authentication: JWT and OAuth2 Basics; CORS, Middleware, Request Lifecycle.
- **3.7 Testing and Quality:** Testing with pytest and FastAPI TestClient, Structuring an API Project for Maintainability, Applied Mini-Project (optionally serving a simple AI model).

Unit-4.

- **4.1 Project Organization:** Folder Structure, Configuration Files, Environment Variables, .env and python-dotenv, Secrets Handling, Twelve-Factor App Principles.
- **4.2 Version Control:** Git Basics, GitHub, Branching, Pull Requests, Collaboration, .gitignore and Commit Hygiene.
- **4.3 Linux Essentials:** File System Commands, Permissions, Process Management.
- **4.4 Deployment Fundamentals:** WSGI vs ASGI, Gunicorn, Uvicorn / Gunicorn Workers, Nginx Basics, Reverse Proxy, Domain Configuration, HTTPS and SSL, Logging and Monitoring.
- **4.5 Containerization:** Introduction to Docker, Images and Containers, Dockerizing Flask and FastAPI Applications, Docker Compose (app + database).
- **4.6 CI/CD (Introductory):** Automated Testing and Deployment with GitHub Actions.
- **4.7 Capstone Project:** Development and Deployment of a Database-Driven Web Application / REST API — a FastAPI backend (with a Flask app or simple frontend as client), containerized, deployed behind Nginx with HTTPS, and version-controlled with a CI pipeline.